2D Vector FRQ Cheat Sheet

Built around the patterns from: - `Exam 2 Review - Spring 2026.pdf` - `Final Exam Sample.pdf` struct-processing FRQ

1. Core facts

- Type: `vector<vector<T>>`
- Rows: `mat.size()`
- Cols in row `r`: `mat.at(r).size()`
- Rectangular matrix: every row has same length
- Square matrix: rows = cols
- Empty matrix check: `mat.empty()`
- Empty row check: `mat.at(r).empty()`

```
for (int r = 0; r < mat.size(); r++) {
    for (int c = 0; c < mat.at(r).size(); c++) {
        // use mat.at(r).at(c)
    }
}
```

2. Most common FRQ patterns

Read every element

```
for (int r = 0; r < mat.size(); r++) {
    for (int c = 0; c < mat.at(r).size(); c++) {
        sum += mat.at(r).at(c);
    }
}
```

Build a new matrix row-by-row

```
vector<vector<int>> out;
for (int r = 0; r < input.size(); r++) {
    vector<int> row;
    for (int c = 0; c < ...; c++) {
        row.push_back(...);
    }
    out.push_back(row);
}
```

Modify in place

```
for (int r = 0; r < mat.size(); r++) {
    for (int c = 0; c < mat.at(r).size(); c++) {
        mat.at(r).at(c) = ...;
    }
}
```

3. Index meaning

- `mat.at(r).at(c)` = row `r`, column `c`
- Same row: `r` fixed, `c` changes
- Same column: `c` fixed, `r` changes
- Main diagonal: `r == c`
- Other diagonal in square `n x n`: `r + c == n - 1`

4. Board / tic-tac-toe scans

Check one row

```
bool same = true;
for (int c = 1; c < board.at(r).size(); c++) {
    if (board.at(r).at(c) != board.at(r).at(0)) same = false;
}
```

Check one column

```
bool same = true;
for (int r = 1; r < board.size(); r++) {
    if (board.at(r).at(c) != board.at(0).at(c)) same = false;
}
```

Check diagonals in square board

```
for (int i = 0; i < n; i++) mainDiag &= (b.at(i).at(i) == target);
for (int i = 0; i < n; i++) otherDiag &= (b.at(i).at(n - 1 - i) == target);
```

Use this for: - tic-tac-toe winner checks - diagonal sums - diagonal counts

5. Neighbor problems

Used in average-matrix questions.

```
double sum = 0;
int count = 0;
for (int dr = -1; dr <= 1; dr++) {
    for (int dc = -1; dc <= 1; dc++) {
        if (dr == 0 && dc == 0) continue;
        int nr = r + dr;
        int nc = c + dc;
        if (nr >= 0 && nr < in.size() &&
            nc >= 0 && nc < in.at(nr).size()) {
            sum += in.at(nr).at(nc);
            count++;
        }
    }
}
out.at(r).at(c) = sum / count;
```

Key idea: - corners have fewer neighbors - edges have fewer neighbors - center may have 8 neighbors

6. Path / adjacency matrix pattern

From `pathLength(distance, path)`:

```
double total = 0;
for (int i = 0; i + 1 < path.size(); i++) {
    total += distance.at(path.at(i)).at(path.at(i + 1));
}
return total;
```

Interpretation: - `path[i]` = current city - `path[i+1]` = next city - matrix entry gives distance between them

7. Row filtering pattern

Example: remove all-zero rows.

```
vector<vector<int>> out;
for (int r = 0; r < mat.size(); r++) {
    bool keep = false;
    for (int c = 0; c < mat.at(r).size(); c++) {
        if (mat.at(r).at(c) != 0) keep = true;
    }
    if (keep) out.push_back(mat.at(r));
}
```

If prompt says throw on empty matrix or empty row, check that first.

8. Struct-processing pattern

From the final sample:

```
vector<vector<int>> out;
bool haveRow = false;
int shortest = 0;

for (int i = 0; i < input.size(); i++) {
    if (input.at(i).min > input.at(i).max) continue;

    vector<int> row;
    for (int x = input.at(i).min; x <= input.at(i).max; x++) {
        row.push_back(x);
    }

    if (!haveRow || row.size() < shortest) {
        shortest = row.size();
        haveRow = true;
    }
    out.push_back(row);
}

for (int r = 0; r < out.size(); r++) {
    while (out.at(r).size() > shortest) out.at(r).pop_back();
}
```

Order: 1. skip invalid bounds 2. build each row 3. track shortest valid row 4. trim all rows to rectangular shape

9. Common mistakes

- Mixing up rows and columns
- Assuming every matrix is square
- Using `mat.at(0).size()` for every row when rows may differ
- Forgetting edge/corner checks in neighbor problems
- Modifying input when prompt wants a new matrix
- Forgetting to initialize `out` to correct shape before writing with `.at()`

10. Safe exam skeletons

Create same-size output matrix

```
vector<vector<double>> out(in.size());
for (int r = 0; r < in.size(); r++) {
    out.at(r) = vector<double>(in.at(r).size());
}
```

Throw checks

```
if (mat.empty()) throw invalid_argument("bad matrix");
for (int r = 0; r < mat.size(); r++) {
    if (mat.at(r).empty()) throw invalid_argument("bad matrix");
}
```

Mental checklist

- read only, modify in place, or return new matrix?
- rectangular or square?
- row loop outside, column loop inside?
- need row condition, column condition, or neighbor condition?
- need to skip rows, trim rows, or build rows?